

KV Store

Esercitazione integrata

Architetture dei Sistemi Distribuiti

Lezione 18

L'esercitazione mette insieme due avanzamenti forti:

- topologia dinamica con `ADD_SHARD` e `REBALANCE`;
- scritture condizionali con `GETV` e `CAS`.

L'obiettivo non e' solo "aggiungere feature". L'obiettivo e' dimostrare di saper progettare un'interfaccia che resti coerente quando:

- il dato si sposta;
- i client osservano versioni;
- piu' operazioni competono sullo stesso stato.

Dalle lezioni precedenti avete gia' questi ingredienti:

- routing su piu' shard;
- comando WHERE per rendere osservabile il target;
- migrazione esplicita;
- versioni per chiave;
- CAS con fallimento osservabile.

L'esercitazione richiede di combinarli in modo coerente, non di tenerli separati.

La cartella della capstone contiene una soluzione di riferimento:

- shard versionato con coppie (`value`, `version`);
- router con `GETV`, `CAS`, `ADD_SHARD` e `REBALANCE`;
- client interattivo;
- test end-to-end automatico.

Non e' il punto di arrivo definitivo. E' un oggetto tecnico da eseguire, leggere e criticare.

Perche' Questa E' Un'Esercitazione Di Progetto

Qui non basta scrivere codice che “passa qualche prova”. Vi sto chiedendo di dimostrare che sapete:

- scegliere una semantica;
- documentarla in modo difendibile;
- implementarla senza contraddirla;
- costruire test che verifichino proprio quella semantica.

Il prodotto finale va letto come un piccolo sistema distribuito con specifica.

Il sistema finale deve almeno supportare:

- GET <key>
- GETV <key>
- SET <key> <value...>
- CAS <key> <expected_version> <value...>
- WHERE <key>
- ADD_SHARD <id> <host> <port>
- REBALANCE

Se aggiungete altro, dovete anche specificarne la semantica.

Contratto Della Reference Implementation

La soluzione inclusa nel lab sceglie esplicitamente:

- versione locale alla chiave;
- stato autorevole conservato nello shard;
- router responsabile di topologia e migrazione;
- ADD_SHARD visibile subito;
- REBALANCE stop-the-world dal punto di vista del router.

La finestra tra ADD_SHARD e REBALANCE resta osservabile: una chiave esistente puo' temporaneamente risultare NOT_FOUND.

Prima del codice dovete fissare alcune decisioni di contratto:

- che cosa rappresenta esattamente la versione?
- la versione sopravvive alla migrazione?
- cosa vede il client tra `ADD_SHARD` e `REBALANCE`?
- una CAS letta prima della migrazione resta valida dopo la migrazione?

Se non decidete queste cose prima, il codice verra' contraddittorio.

La specifica che voglio vedere dovrebbe chiarire almeno:

- sintassi dei comandi e forma delle risposte;
- significato preciso di `version`;
- comportamento osservabile di `GET`, `GETV`, `CAS`;
- cosa accade durante e dopo `REBALANCE`;
- quali casi il sistema dichiara esplicitamente fuori contratto.

Se una di queste parti manca, l'interfaccia non e' ancora davvero specificata.

Il sistema deve rispettare almeno questi vincoli:

- una migrazione non deve perdere ne' valore ne' versione;
- una CAS con versione stantia deve fallire;
- una CAS con versione corretta deve aggiornare sia valore sia versione;
- dopo REBALANCE, routing e posizione reale devono tornare coerenti.

Questi non sono suggerimenti. Sono il nucleo dell'accettazione.

Protocollo Di Migrazione Implementato

La reference implementation usa un protocollo volutamente semplice:

- ① il router raccoglie gli item da tutti gli shard;
- ② per ogni chiave ricalcola il target con la topologia corrente;
- ③ se serve, invia `IMPORT_KEY(value, version)`;
- ④ solo dopo invia `DELETE_LOCAL(version)` allo shard sorgente.

Il dettaglio importante e':

value e version viaggiano insieme.

Milestone 1

Primo obiettivo: rendere solido lo store versionato senza migrazione.

- 1 implementare GETV;
- 2 implementare CAS;
- 3 dimostrare un caso di successo;
- 4 dimostrare un `version_mismatch`.

Prima di parlare di shard, dovete poter difendere la semantica locale della versione.

Secondo obiettivo: reintrodurre la topologia dinamica.

- ➊ aggiungere uno shard;
- ➋ rendere visibile il nuovo routing;
- ➌ osservare la divergenza tra target teorico e posizione reale;
- ➍ eseguire il rebalance.

Qui si misura se avete capito che il routing non esaurisce il significato del sistema.

Terzo obiettivo: combinare i due problemi.

- una chiave puo' cambiare shard;
- la sua versione non deve perdere significato;
- una CAS preparata prima dello spostamento deve comportarsi secondo contratto anche dopo lo spostamento.

Questa e' la parte davvero interessante: non state solo migrando dati, state migrando *stato osservabile*.

Prima di implementare, ogni gruppo dovrebbe saper rispondere a domande come:

- il router deve sapere anche dove la chiave *sta*, oltre a dove *dovrebbe stare*?
- la versione vive nello shard o nel router?
- una migrazione trasferisce valore, versione o entrambi piu' metadata?
- una CAS puo' essere eseguita mentre la migrazione e' in corso?

Il codice viene dopo queste risposte, non prima.

Avete almeno due direzioni progettuali:

- **router intelligente**: il router conosce topologia, migrazione e inoltro delle operazioni versionate;
- **shard piu' ricchi**: ogni shard conserva coppie (value, version) e il rebalance trasferisce entrambe.

In entrambi i casi dovete spiegare:

- dove vive la versione;
- chi ne garantisce la conservazione;
- chi decide la validita' di una CAS.

Router piu' intelligente

- facilita forwarding e osservabilita' del transitorio;
- accentra molto potere decisionale;
- rischia di diventare il punto piu' fragile del progetto.

Shard piu' autonomi

- separazione piu' pulita tra routing e stato della chiave;
- protocollo di migrazione piu' importante;
- richiede piu' disciplina nel trasporto di valore e versione.

Una squadra ben organizzata potrebbe separare:

- definizione del contratto e dei casi limite;
- estensione degli shard per gestire (`value`, `version`);
- estensione del router per routing e migrazione;
- campagna di test e scenari di validazione.

Non e' una divisione rigida, ma evita il problema classico: ognuno implementa un pezzo diverso con semantiche incompatibili.

Una sequenza pragmatica e':

- ① fissare il contratto di GETV, CAS e chiave assente;
- ② trasferire correttamente (`value`, `version`) durante REBALANCE;
- ③ definire il significato di WHERE prima, durante e dopo la migrazione;
- ④ decidere il comportamento di CAS nella finestra transitoria;
- ⑤ solo alla fine lavorare su trasparenza e rifiniture.

Questo ordine riduce il rischio di costruire un sistema grande ma semanticamente confuso.

Test Di Accettazione Minimi

1. SET k v0
2. GETV k -> versione iniziale
3. CAS k versione v1 -> successo
4. ADD_SHARD ...
5. REBALANCE
6. GETV k -> stessa storia, shard diverso
7. CAS k vecchia v2 -> mismatch

Se questo scenario non e' chiaro o non e' ripetibile, il sistema non e' ancora pronto.

Il lab include un test di accettazione:

```
python3 labs/kv_store/capstone_exercise/acceptance_test.py
```

Il test verifica:

- CAS riuscita;
- `version_mismatch`;
- cambio di routing dopo `ADD_SHARD`;
- finestra `NOT_FOUND` prima di `REBALANCE`;
- conservazione della versione dopo migrazione.

Per un gruppo che parte dai laboratori precedenti, una stima ragionevole puo' essere:

- 30–45 minuti per contratto e casi critici;
- 45–60 minuti per migrazione di valore e versione;
- 30–45 minuti per integrare CAS nel percorso shardato;
- 30–45 minuti per test e nota tecnica finale.

Il punto non e' la precisione al minuto. Il punto e' capire che una parte importante del tempo va spesa in decisioni e verifica, non solo in scrittura di codice.

Oltre ai test minimi, mi aspetto una vera campagna di validazione:

- test nominali su chiave singola;
- test di conflitto con due client;
- test di rebalancing senza scritture concorrenti;
- test post-migrazione con GETV e CAS;
- se possibile, almeno un test su finestra transitoria.

Un buon test plan vale quasi quanto una buona implementazione.

Gli errori che mi aspetto di vedere emergere sono:

- migrare il valore ma non la versione;
- resettare la versione quando la chiave cambia shard;
- non sapere chi decide la validita' di una CAS durante il transitorio;
- promettere con WHERE piu' di quanto il sistema sappia davvero garantire.

Questi errori non sono dettagli. Rompono il contratto proprio nei punti piu' interessanti.

Errori Tipici Che Voglio Vedere Evitati

- migrare il valore ma non la versione;
- resettare la versione quando la chiave cambia shard;
- usare `WHERE` come se implicasse già' la presenza reale del dato;
- trattare `version_mismatch` come errore tecnico invece che come esito del contratto;
- non scrivere test per la finestra transitoria.

Non mi aspetto un sistema perfetto. Mi aspetto trade-off dichiarati con onesta':

- disponibilita' contro pulizia semantica;
- semplicita' implementativa contro trasparenza delle letture;
- localita' della versione contro coordinamento globale;
- protocolli brevi contro gestione robusta dei guasti.

Se scegliete una soluzione debole ma la motivate bene, e' meglio di una soluzione "forte" non compresa.

Ogni gruppo deve consegnare:

- ① una specifica breve dell'interfaccia;
- ② un'implementazione funzionante;
- ③ test o procedura ripetibile di verifica;
- ④ una nota tecnica sui limiti rimasti aperti.

La nota tecnica conta molto: se il vostro sistema ha limiti dichiarati e coerenti, vale piu' di una soluzione che “sembra funzionare” ma non sa dire che cosa promette.

Valutero' soprattutto:

- chiarezza del contratto;
- correttezza dei casi nominali;
- qualita' dei test sui casi critici;
- capacita' di motivare trade-off e limiti;
- coerenza tra interfaccia dichiarata e comportamento reale.

Che Cosa Fa Scendere Molto La Valutazione

- interfaccia implementata ma non specificata;
- slide o README che promettono piu' di quanto il codice faccia davvero;
- versioni perse o resettate in migrazione;
- test solo "felici", senza casi critici;
- incapacita' di spiegare il comportamento del sistema negli stati intermedi.

Questa esercitazione non chiede solo di costruire un sistema piu' grande. Chiede di dimostrare maturita' progettuale:

- separare meccanismo e contratto;
- ragionare su stati transitori;
- rendere difendibili le promesse dell'interfaccia.

Se il codice funziona ma non sapete spiegare cosa promette nei casi difficili, il lavoro non e' completo.